

KnackELT Architecture

How knack-elt works, and where it sits in a full Knack → warehouse → BI stack.

Four diagrams:

1. [Reference architecture](#) — the end-to-end stack, in [plain language](#) and with [the technical detail](#)
2. [Pipeline flow](#) — what knack-elt actually does on a run
3. [Run sequence](#) — the call order for one invocation
4. [SCD2 row lifecycle](#) — how history is represented, and how to query it

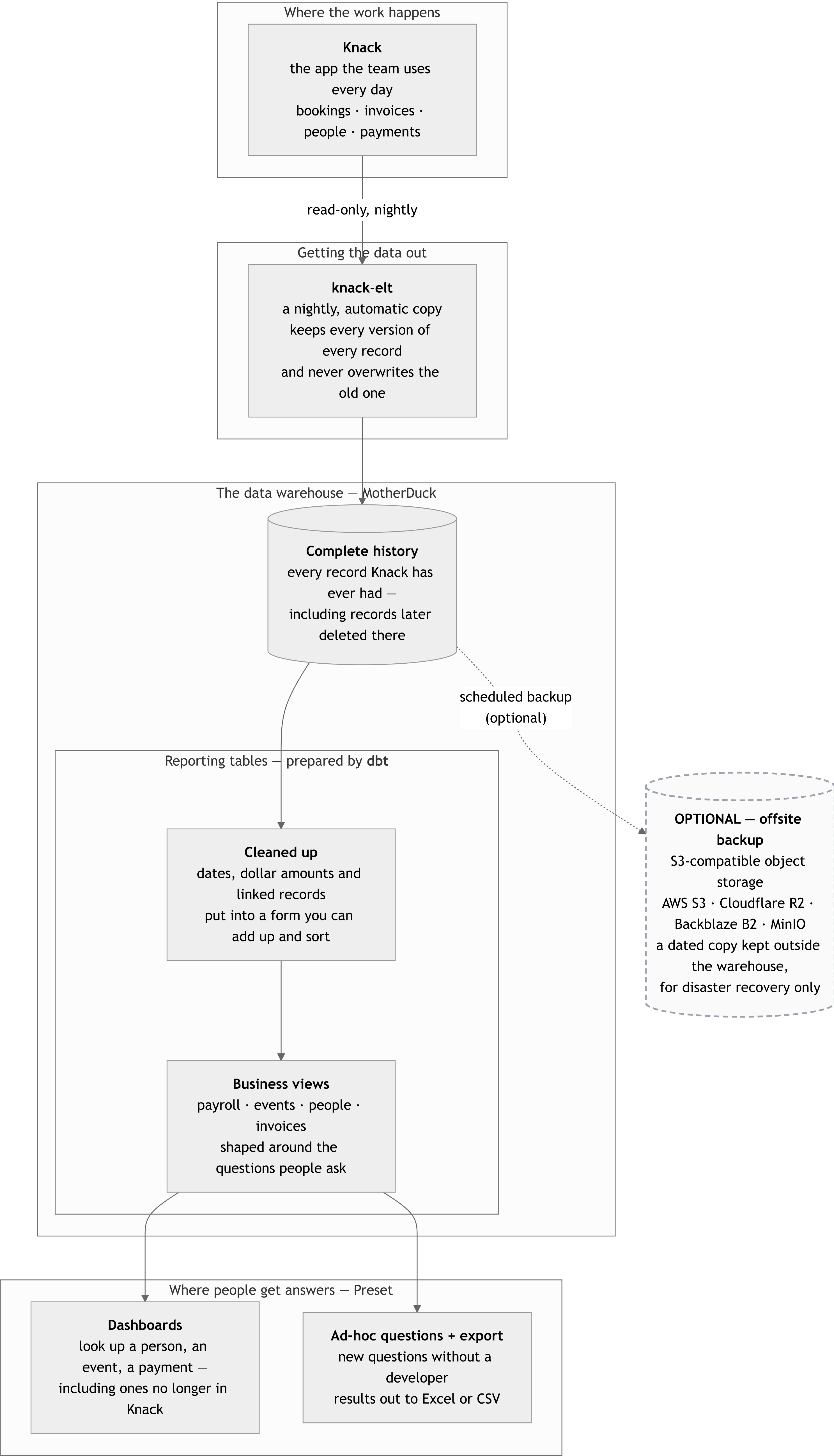
Scope note. This repo is the **ingestion** piece only. The dbt models, BI configuration, and CI orchestration shown in diagram 1 are drawn from a working production deployment where they live in a companion application repo (whose `pipelines/knack_dlt.py` is the production twin of this repo's pipeline). They are real, not aspirational — but they are not in this tree. Subgraph labels mark what lives where.

1. Reference architecture

Knack is the system of record and the operational UI. It is not a warehouse: no SQL, no aggregates, no history, and a record cap that eventually forces pruning. The stack below turns it into one — MotherDuck as the warehouse, dbt as the transformation layer, Preset (managed Apache Superset) as the access layer.

Same architecture, drawn twice: once for a stakeholder audience, once with the detail an engineer needs.

The stack in plain language

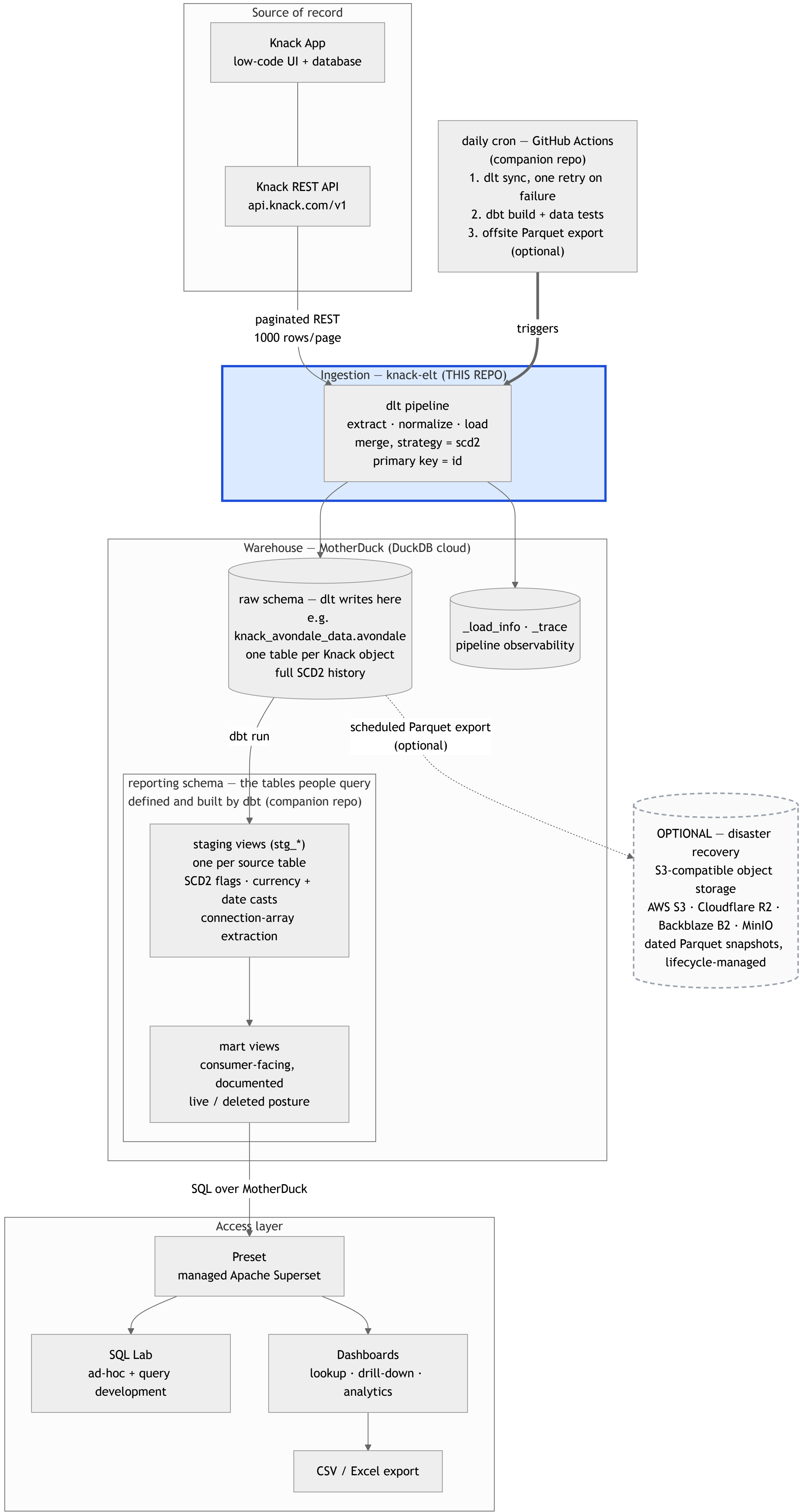


How to read it. The copy runs one way only — nothing this stack does can change data in Knack. The warehouse keeps history, so a record deleted in Knack is still answerable afterwards; that is the whole point of having it. The two boxes inside the warehouse are a division of labour, not two copies of the data: the complete history is what arrives, and **dbt** is the tool that turns it into the cleaned, business-shaped tables people actually query.

The dashed box is genuinely optional. The warehouse is already a second copy of Knack, so this is a third — insurance against losing the *warehouse* (a bad migration, an account problem, a provider outage), not against losing Knack. Add it when the warehouse becomes the only remaining copy of pruned records; skip it while Knack still holds everything. Any S3-compatible store works, and retention is normally handled by the bucket's own lifecycle rules rather than by code.

The same stack, with the technical detail

FIGURE 2 — THE SAME STACK, WITH THE TECHNICAL DETAIL



The load-bearing detail: schema ownership is split.

Schema	Owner	Contents	Rule
raw (e.g. <code>avondale</code>)	dlt	One table per Knack object, full SCD2 history, Knack-shaped values	dbt never builds into it. It is the pipeline's output, and a <code>dbt run</code> writing here would be clobbered on the next sync.
<code>reporting</code>	dbt	Staging + mart views	Everything downstream reads here. No dashboard queries raw directly.

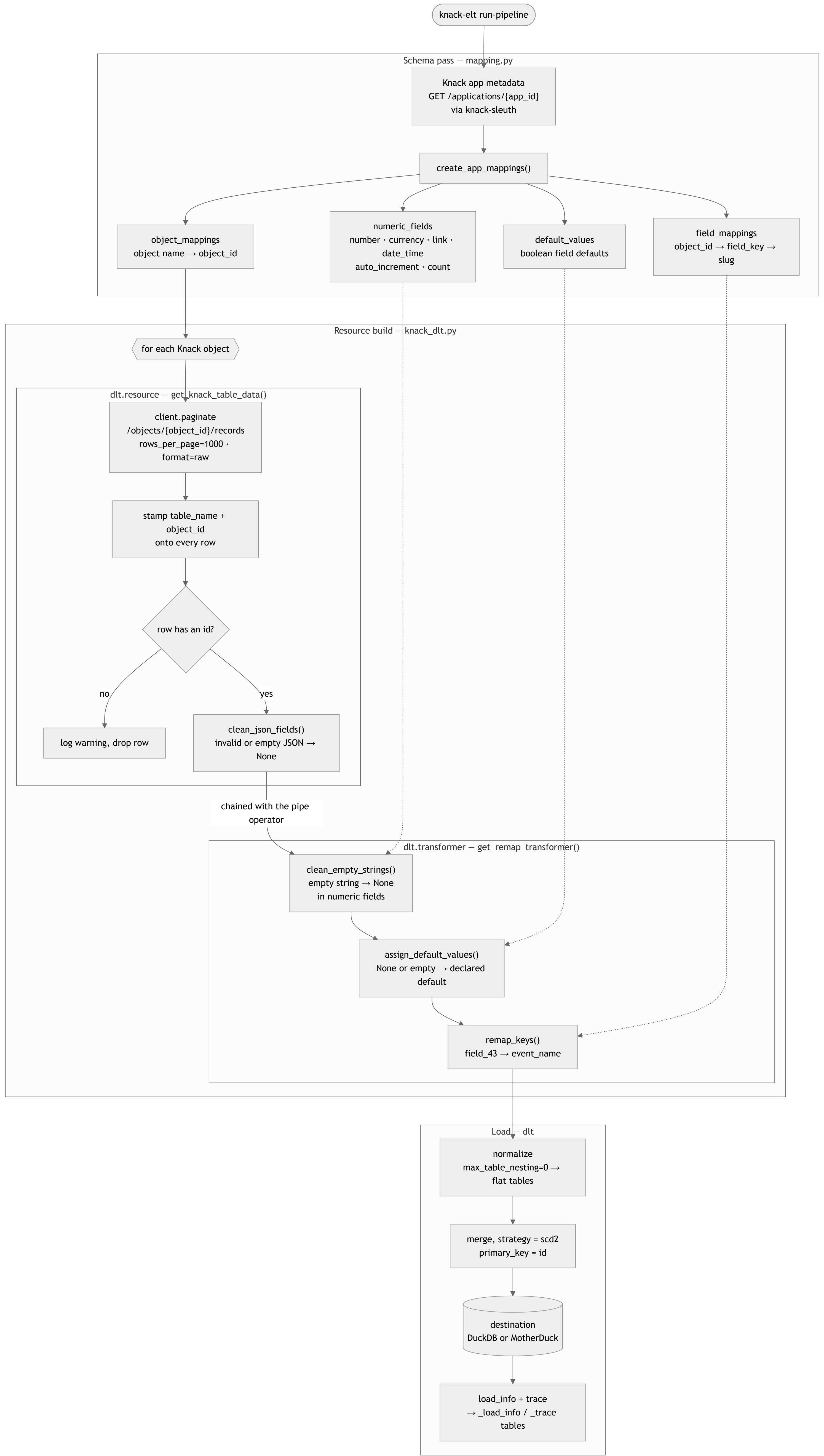
Staging exists so that exactly one layer absorbs the mess Knack and dlt produce together — currency strings with commas, dates wrapped in JSON, connection fields as JSON arrays, and dlt's type-variant columns (when a value doesn't fit the type inferred from the first batch, dlt NULLs the base column and diverts the value to a sibling like `amount__v_double`, which makes a naive `SUM()` under-report silently). Marts and dashboards then get to be simple.

Why Preset: it is managed Superset, so SQL Lab doubles as the query-development environment and the dashboard builder against the same MotherDuck connection — a proven query becomes a dataset becomes a chart with no rewrite. Any BI tool with a DuckDB/MotherDuck SQLAlchemy connection substitutes here; the contract is the `reporting` schema, not the tool. Note that row-level security and embedded analytics are paid-tier features in Preset — untrusted multi-tenant self-service needs a different surface (a scoped API in front of MotherDuck), not a BI seat.

2. Pipeline flow

What one run of knack-elt does. Two inputs from Knack: the **app metadata** (schema) and the **records** (data). The metadata is what turns `field_73` into `payroll_name`.

FIGURE 3 — PIPELINE FLOW



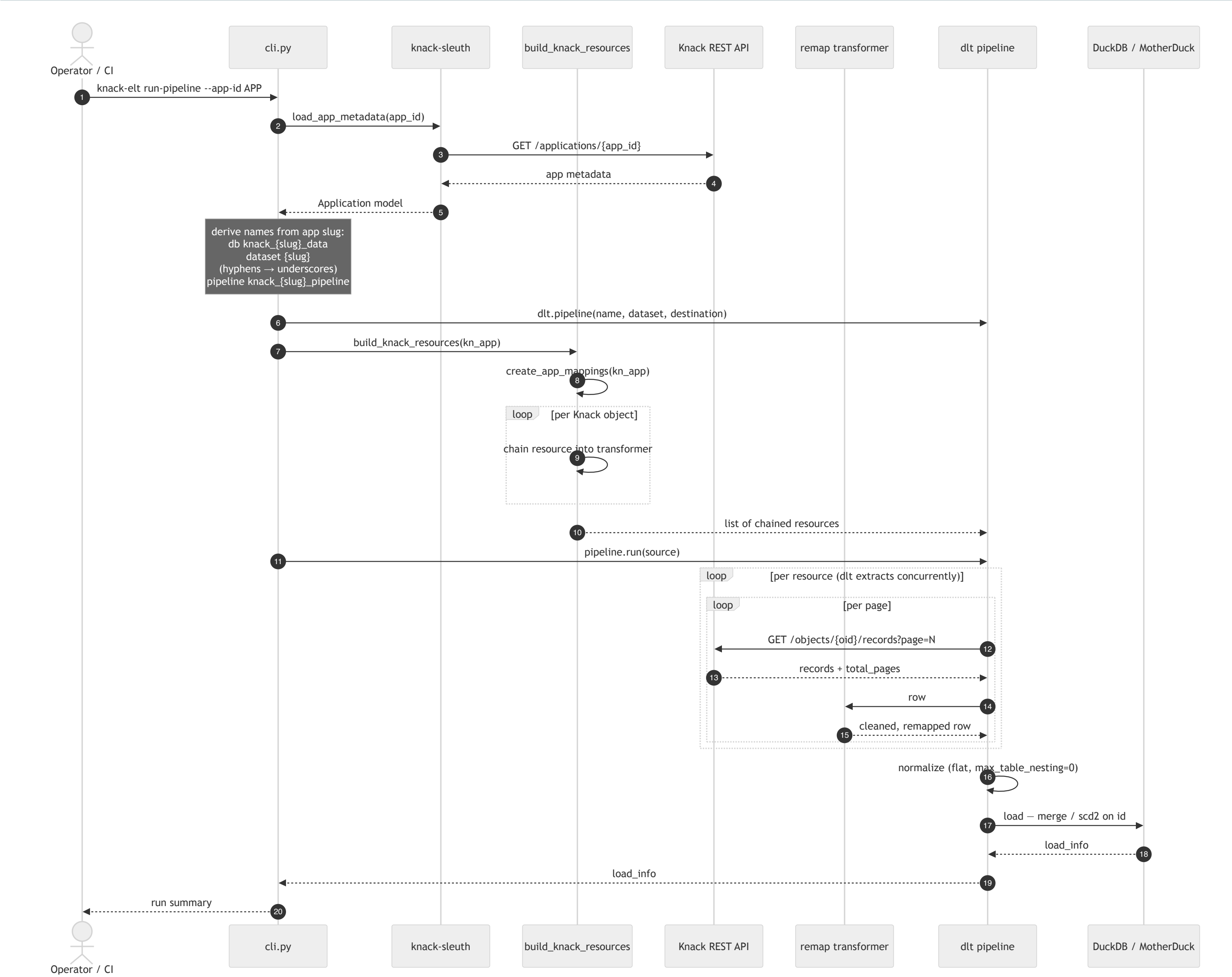
Notes on the mapping pass

- Field names are slugified with `re.sub(r'^a-z0-9+', '_', name.lower()).strip('_')`, so "Vocalist / Song Details" becomes `vocalist_song_details`.
 - A user-defined Knack field literally named `id` would collide with Knack's own record id, so it is renamed to `<singular>_id` (e.g. `worker_id`). Knack's row id is always `id`, and it is the pipeline's primary key.
 - `numeric_fields` and `default_values` are registered under the field key, the raw name, **and** the slug, because cleaning runs before and after the remap.
 - `remap_keys` falls back to the original key when no mapping exists, so an unmapped object still loads — with `field_NN` column names.
-

3. Run sequence

The packaged entry point is the Typer CLI (`knack-elt run-pipeline`). It takes the app metadata from `knack-sleuth` rather than fetching it itself.

FIGURE 4 — RUN SEQUENCE



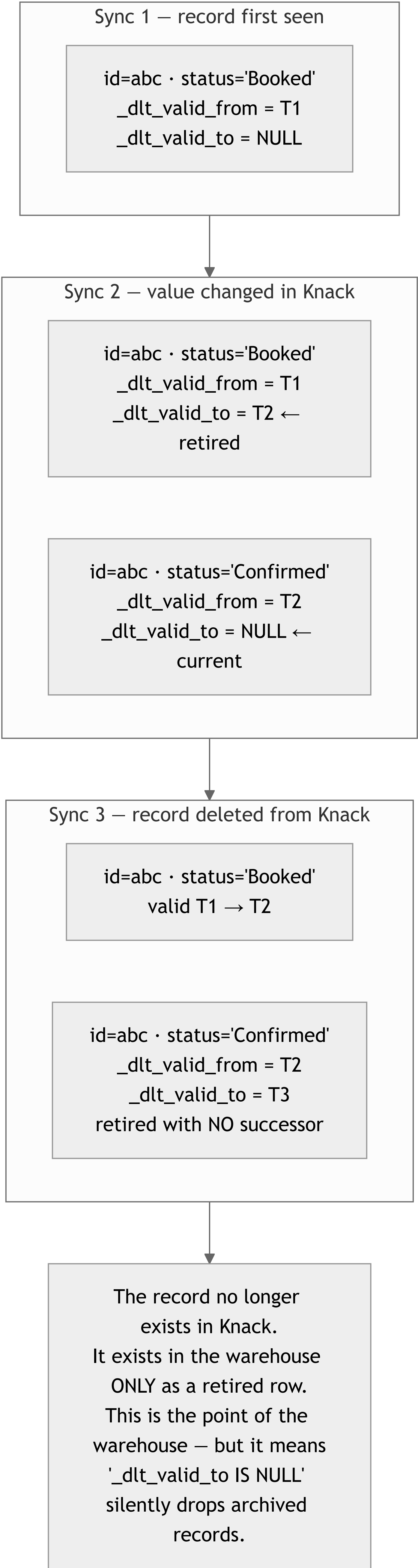
Two entry points, and they do not target the same destination today:

Entry point	Metadata source	Object list	Destination
<code>cli.py</code> <code>run_pipeline</code> (<code>knack-elt run-pipeline</code>)	<code>knack-sleuth</code> <code>load_app_metadata()</code>	every object in the app	local DuckDB at <code>tests/data/knack_{slug}_data.duckdb</code> — the MotherDuck destination is present but commented out (<code>cli.py:80-84</code> , see the TODO at <code>cli.py:68</code>)
<code>python -m</code> <code>knack_elt.knack_dlt</code> (<code>knack_ave_source</code>)	its own HTTP GET <code>/applications/{id}</code>	every object in the app	MotherDuck , database name hardcoded to <code>knack_avondale_data</code>

The `__main__` path is the older one; it also sets `load.workers = 3` , `truncate_staging_dataset = True` , exports the schema to `pipelines/schemas/export` , and writes `load_info` and the run trace back to the destination as `_load_info` and `_trace` . Making the destination a CLI flag is the obvious next step and the TODO already names it.

4. SCD2 row lifecycle

Every table is loaded with `write_disposition={"disposition": "merge", "strategy": "scd2"}` , so the warehouse is **append-and-retire, not overwrite**. dlt adds `_dlt_valid_from` and `_dlt_valid_to` to every row. A record's history is therefore several rows sharing one `id` .



Because of that last case, two different flags are needed, and conflating them is the most common way to get a wrong answer out of this stack:

```
-- one row per record, live or deleted, plus the live/deleted flag
with flagged as (
  select
    *,
    row_number() over (partition by id order by _dlt_valid_from desc) = 1
      as latest_version,
    _dlt_valid_to is null as is_live_in_knack
  from avondale.some_table          -- the raw, dlt-owned schema
)
select * from flagged where latest_version
```

Question	Filter
Current state of everything still in Knack	latest_version and is_live_in_knack
Latest known state of every record ever seen, including deleted	latest_version
What did this record look like on a given date	_dlt_valid_from <= d and (_dlt_valid_to is null or _dlt_valid_to > d)
What has been deleted from Knack	latest_version and not is_live_in_knack

Two traps worth stating explicitly:

- **Partition by `id`, not by a Knack field that looks like an id.** `id` is the canonical Knack record id supplied by the API and is populated on every row. A user-added "Record ID" *field* is NULL on rows created before it existed, and partitioning on it collapses all of them into one NULL group and corrupts the flags.
- **Aggregate without `latest_version` and you double-count.** Every historical version of a row is still a row. A `SUM()` over the raw table sums every version of every record.

Deleted-in-Knack is not the same as *archived by policy* — distinguishing an intentional retention prune from incidental deletion needs a separate manifest of what the pruning process removed; SCD2 alone cannot tell them apart.

Configuration

`pydantic-settings`, loaded from environment or `.env` (`src/knack_elt/config.py`):

Variable	Used for
KNACK_APP_ID	Knack application id — also the default for <code>--app-id</code>
KNACK_API_KEY	Knack REST API key (sent as <code>X-Knack-REST-API-Key</code>)
motherduck_api_key	MotherDuck token, interpolated into the <code>md:///</code> connection string

Note that dbt-duckdb reads the MotherDuck token from `motherduck_token` , not `motherduck_api_key` — the two layers disagree on the variable name, so a deployment running both needs to set or shim both.